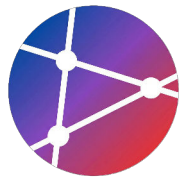


The Influence of Open and Inner Source on Software Delivery Performance

오픈소스와 이너소스가
소프트웨어 전달 성능에 미치는 영향



학습 목표

Learning Objectives for This Session



원칙과 가치 이해

오픈소스의 핵심 원칙(투명성, 협업 등)과 가치를 이해합니다.



라이선스 구분 및 선택

다양한 오픈소스 라이선스(MIT, GPL 등)를 구분하고 적절히 선택합니다.



Inner Source 적용

Inner Source 개념을 이해하고 조직 내 적용 방법을 학습합니다.



프로젝트 기여 방법

오픈소스 프로젝트에 효과적으로 기여하는 절차와 방법을 습득합니다.



성과 영향 측정

오픈소스와 Inner Source가 소프트웨어 전달 성능(DORA metrics)에 미치는 영향을 이해합니다.



SECTION 01

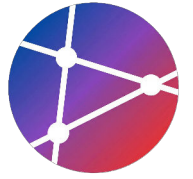


오픈소스란?

오픈소스의 정의, 역사, 통계를 다룹니다.

오픈소스 정의

What is Open Source Software (OSS)?

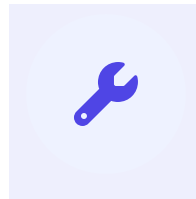


Open Source Software (OSS)란,
소스 코드가 공개되어 누구나 자유롭게 접근하여 사용할 수 있는 소프트웨어를 의미합니다.



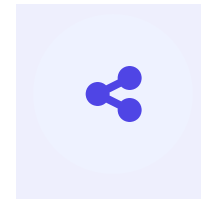
사용 (Use)

개인적 목적이나 상업적 목적에
관계없이 자유롭게
소프트웨어를 실행할 수 있습니다.



수정 (Modify)

공개된 소스 코드를 분석하고,
자신의 필요에 맞게 기능을
개선하거나 변경할 수 있습니다.



배포 (Distribute)

원본 소프트웨어 또는 수정된
버전을 타인에게 자유롭게
복제 및 전달할 수 있습니다.

오픈소스의 역사

Evolution of Open Source Software



🏠 1950s - 1960s

공유의 시대

소스 코드 공개가 학계와 연구소의 기본 문화로 자리잡음

🔒 1970s - 1980s

상업적 소프트웨어 등장

저작권 강화, 소스 비공개(Closed Source) 확산

🌐 1983

GNU Project

Richard Stallman의 자유 소프트웨어(Free Software) 운동

🐧 1991

Linux Kernel

Linus Torvalds의 리눅스 커널 공개 및 협업 모델 성공

💡 1998

"Open Source" 용어 탄생

이념적 부담을 줄이고 실용성을 강조하며 용어 공식 채택

🌀 2000s

생태계의 폭발적 성장

Apache, MySQL, PHP 등 웹 기술 발전과 함께 급성장

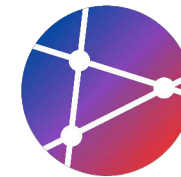
🌐 2020s ~ Present

오픈소스의 보편화

AI, Cloud 등 거의 모든 최신 기술이 오픈소스 기반

오픈소스 통계

Global Impact & Market Growth (2024)



GitHub 2024

100M+

Total Developers

300M+

Repositories Hosted

대부분의 프로젝트가 오픈소스



\$32B

↑ 25%

Global Market Size (2023)



95%

Enterprise Adoption



SECTION 02

오픈소스의 핵심 가치

투명성·협업·혁신·품질·커뮤니티의 5가지 핵심 가치를 심도 있게 탐구합니다.



02



| 핵심 가치: 투명성 (Transparency)

투명성이 가져오는 신뢰와 품질의 선순환 구조



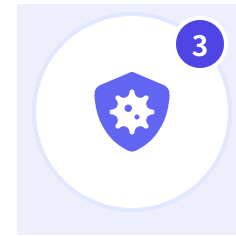
코드 공개

소스 코드가 외부에
투명하게 공개됨



누구나 검토

전 세계 개발자들이
코드를 자유롭게 분석



취약점 발견

보안 허점과 버그를
조기에 식별하고 수정



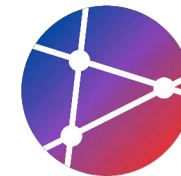
신뢰 구축

검증된 안전성으로
사용자 신뢰 확보

“ 많은 눈이 있으면, 모든 버그는 얕아진다 (Given enough eyeballs, all bugs are shallow)” —
Linus's Law ”

핵심 가치: 협업

Core Value: Collaboration



01

전 세계 개발자 협력

지리적 경계를 넘어 전 세계 수백만 명의 개발자들이 하나의 프로젝트에 기여하며 강력한 생태계를 구축합니다.



02

다양한 관점

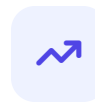
서로 다른 배경과 경험을 가진 기여자들의 참여로 창의적인 아이디어와 다각적인 해결책이 제시됩니다.



03

빠른 문제 해결

'충분한 눈이 있다면 모든 버그는 얄다'는 원칙처럼, 다수의 검토를 통해 이슈를 신속하게 식별하고 해결합니다.



04

지속적 개선

커뮤니티의 지속적인 피드백과 코드 기여(Pull Requests)를 통해 소프트웨어가 끊임없이 진화하고 발전합니다.



핵심 가치: 혁신 (Innovation)

오픈소스는 제약 없는 실험을 통해 산업 발전을 가속화합니다.



제약 없는 실험

실패에 대한 비용이나 제약 없이 자유롭게 기술적인 시도를 할 수 있는 환경을 제공하여 창의성을 극대화합니다.



새로운 아이디어

다양한 배경을 가진 개발자들의 관점이 융합되어 기존의 방식에 얽매이지 않는 혁신적인 아이디어를 도출합니다.



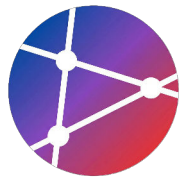
빠른 프로토타이핑

공개된 소스 코드를 재사용하고 조합하여 아이디어를 신속하게 구현(PoC)하고 검증할 수 있습니다.



산업 발전

개별 프로젝트의 혁신이 모여 기술 표준을 형성하고, 전체 소프트웨어 산업의 기술 수준을 한 단계 끌어올립니다.



핵심 가치 4. 품질 (Quality)

Open collaboration leads to superior software quality

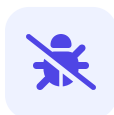
"Given enough eyeballs, all bugs are shallow."

— LINUS'S LAW (ERIC S. RAYMOND)



많은 눈의 검토

전 세계의 수많은 개발자가 코드를 검토함으로써 사각지대를 최소화하고 투명성을 확보합니다.



버그 조기 발견

다양한 환경에서 실행되고 테스트되므로, 숨겨진 버그와 보안 취약점을 빠르게 찾아낼 수 있습니다.



모범 사례 공유

업계의 최신 기술 트렌드와 아키텍처 패턴이 반영되어 자연스럽게 코드의 품질 표준이 높아집니다.

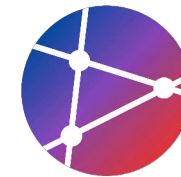


높은 안정성

지속적인 리팩토링과 개선 과정을 통해 상용 소프트웨어 이상의 안정성과 신뢰성을 제공합니다.

핵심 가치: 커뮤니티

The Power of People: Building a Sustainable Ecosystem



공동체 형성

단순한 코드가 아닌, 공통의 목표와 가치를 공유하는 사람들의 모임입니다.



지식 공유

문제 해결 과정과 기술적 노하우가 투명하게 공유되어 모두가 함께 학습합니다.



멘토링

경험 많은 기여자가 새로운 참여자를 이끌어주며 성장을 돕는 문화가 존재합니다.



네트워킹

전 세계의 다양한 개발자들과 연결되어 새로운 협업 기회를 창출합니다.



건강한 생태계

개인의 기여가 모여 지속 가능한 소프트웨어 발전과 지원 체계를 만듭니다.



SECTION 03

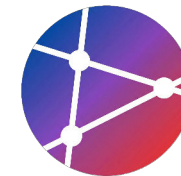


오픈소스 라이선스

라이선스의 역할과 선택 가이드를 이해합니다.

라이선스 선택 가이드

Why Licenses Matter & How to Choose the Right One



라이선스가 정의하는 것

오픈소스 라이선스는 소프트웨어의 사용 조건을 법적으로 정의합니다.
올바른 라이선스 선택을 위해 다음 5가지를 고려해야 합니다.



상업적 사용

제품을 판매할 수 있는가?



수정 권한

코드를 변경할 수 있는가?



재배포

다른 사람에게 전달 가능한가?



소스 공개 의무

내 코드를 공개해야 하는가?

DECISION TREE

Q1. 상업적 사용 허용?

NO

YES

CC-BY-NC
비상업용

Q2. 파생 저작물
공개 의무?

NO

YES

GPL / AGPL
강한 공유

Q3. 특허 보호
필요?

NO

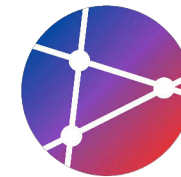
YES

MIT
일반용

Apache
기업용

주요 라이선스 분류

Classification of Major Open Source Licenses



Permissive

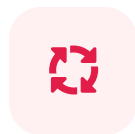
관대한 라이선스

사용, 수정, 배포의 제약이 가장 적으며, 상업적 이용이 자유로운 라이선스입니다.

MIT

Apache 2.0

BSD



Copyleft

강한 공유 라이선스

수정된 코드를 배포할 경우 소스 코드 공개 의무가 발생하는 전염성 라이선스입니다.

GPL

LGPL

AGPL



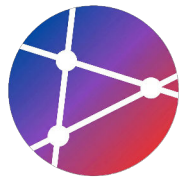
기타 (Others)

파일 단위 / 비소프트웨어

파일 단위로 적용되거나, 소프트웨어가 아닌 문서 및 미디어 저작물에 사용됩니다.

MPL

CC (Creative Commons)



Permissive 라이선스

가장 관대하고 인기 있는 오픈소스 라이선스 분류 (소스 공개 의무 없음)



MIT License

가장 널리 사용되는 심플한 라이선스

- ✓ 상업적 사용, 수정, 배포 자유
- ✓ 사적(Private) 사용 가능
- ❗ 라이선스 및 저작권 고지 필수

대표 사용 사례

React Vue.js Node.js jQuery



Apache 2.0

특허권 보호가 명시된 라이선스

- ✓ MIT와 유사하나 특허 보호 포함
- ✓ 상표권 사용에 대한 명시적 제한
- 📄 기여자 라이선스 동의(CLA) 연계

대표 사용 사례

Android Kubernetes TensorFlow



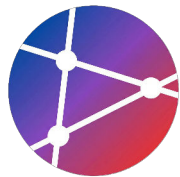
BSD License

극도로 간결하고 제약이 적은 라이선스

- ✓ 내용이 매우 짧고 단순함
- ✓ 제약 사항 최소화 (2-Clause / 3-Clause)
- 🏛️ 학술적 연구 결과물에서 유래

대표 사용 사례

FreeBSD Django Flask Nginx



Copyleft (강한 제약) 라이선스

Strong copyleft licenses requiring source code disclosure



강력한 제약

GPL (General Public License)

- ✓ 상업적 사용, 수정, 배포 가능
- ⚠ 수정 시 전체 소스 공개 의무
- 🔗 파생 저작물도 GPL 전염

EXAMPLES

- 🐧 Linux Kernel (v2)
- 🔧 GNU Tools (v3)



완화된 제약

LGPL (Lesser GPL)

- ✓ GPL보다 조건이 완화됨
- 🛡 동적 링크 시 소스 공개 불필요
- ✍ 라이브러리 자체 수정 시엔 공개

EXAMPLES

- 🖋 Qt (일부 모듈)
- 🖼 GTK+



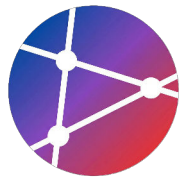
네트워크 제약

AGPL (Affero GPL)

- ↑ GPL보다 가장 강력한 제약
- 🌐 네트워크 서비스(SaaS) 시에도 공개 의무
- 🌐 웹 서비스 기반 소프트웨어 타겟

EXAMPLES

- 🌱 MongoDB (과거 버전)
- ☁ Nextcloud



기타 라이선스

특수한 목적이나 비소프트웨어 저작물을 위한 라이선스 모델



Mozilla Public License (MPL)

Hybrid License (Weak Copyleft)

Permissive와 Copyleft의 중간 지점에 위치한 라이선스로, 프로젝트 전체가 아닌 '**파일 단위**'로 라이선스가 적용되는 것이 특징입니다.

- ✓ **파일 단위 공개:** MPL이 적용된 소스 파일을 수정했을 때만 공개 의무 발생
- ✓ **유연한 결합:** 독점 소프트웨어와 MPL 코드를 함께 링크하여 배포 가능
- 📦 **대표 사례:** Firefox, Thunderbird, LibreOffice



Creative Commons (CC)

For Content, Media & Docs

⚠ 소프트웨어 코드가 아닌 문서, 이미지, 디자인 등 **콘텐츠**에 주로 사용됩니다.



CC0

Public Domain (제약 없음)



CC-BY

저작자 표시 (가장 일반적)



CC-BY-SA

동일조건 변경허락 (위키백과)



CC-BY-NC

비상업적 사용만 허용

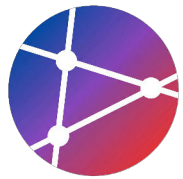


SECTION 04



오픈소스 프로젝트 시작하기

레포지토리 구성과 문서/라이선스 세팅



레포지토리 설정

Setting up a new open source repository with essential files

1 GitHub 레포지토리 생성

GitHub에서 새로운 Public 레포지토리를 생성합니다. 초기화 옵션 (README 등)은 체크하지 않는 것이 깔끔합니다.

2 README.md 파일 작성

프로젝트의 얼굴이 되는 파일입니다. 프로젝트 이름, 설명, 설치 방법, 사용법, 라이선스 정보를 반드시 포함해야 합니다.

3 필수 섹션 포함

`Description`, `Installation`, `Usage` 등 표준 섹션을 준수하세요.



Tip

명령어 하나로 기본 템플릿을 생성하면 시간을 절약할 수 있습니다.

```
bash — README.md Generator

# 2. README.md 작성 (Heredoc 사용)
cat > README.md << 'EOF'
# Project Name
## Description
간단한 프로젝트 설명

## Installation
```bash
npm install project-name
```

## Usage
```javascript
const project = require('project-name');
```

## Contributing
```



LICENSE 파일 추가

Adding an open source license to define usage rights and protect your project

1 GitHub 파일 생성

레포지토리 상단의 `Add file` > `Create new file` 을 클릭합니다.

2 템플릿 마법사 호출

파일 이름을 `LICENSE` 라고 입력하면 우측에 "Choose a license template" 버튼이 나타납니다.

3 라이선스 선택 및 커밋

원하는 라이선스(예: MIT License)를 선택하고 `Review and submit` 을 클릭하여 파일을 생성합니다.

i Note

MIT License는 가장 대중적이며 제약이 적은 관대한(Permissive) 라이선스입니다.

```
LICENSE (MIT Template)

MIT License

Copyright (c) 2025 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
```

문서화: 기여 가이드 및 행동 강령

Setting clear guidelines for contributors and community standards



1 CONTRIBUTING.md 작성

기여 프로세스를 명확히 정의합니다:

- Fork & Clone 방법
- 브랜치 전략 및 커밋 메시지 컨벤션
- PR 생성 절차

2 Code of Conduct (행동 강령)

건강한 커뮤니티를 위한 약속입니다:

- 긍정적 행동(존중, 공감) 장려
- 부정적 행동(괴롭힘, 차별) 금지
- [Contributor Covenant](#) 표준 채택 권장

Best Practice

명확한 가이드는 불필요한 커뮤니케이션 비용을 줄이고, 잠재적 기여자의 참여 장벽을 낮춥니다.

```
markdown — CONTRIBUTING.md Preview

# Contributing to [Project Name]
## 🚀 시작하기
### 1. Fork & Clone
```bash
git clone https://github.com/YOUR-USERNAME/project.git
```

### 2. 브랜치 생성
```bash
git checkout -b feature/amazing-feature
```

### 3. 변경사항 커밋 (Conventional Commits)
```bash
git commit -m "feat: add amazing feature"
```

# Code of Conduct
## 우리의 서약
우리는 모두에게 열린, 환영하는, 다양한, 포용적이고 건강한
커뮤니티를 만들 것을 서약합니다.

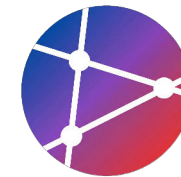
## 우리의 기준
```



SECTION 05

오픈소스 기여하기

기여 유형, 첫 기여 탐색, PR 예시



기여 유형: 코드 기여

Code Contributions: The Foundation of Open Source



MOST COMMON

버그 수정 (Bug Fixes)

기존 코드의 오류를 식별하고 수정하여 소프트웨어의 안정성과 신뢰성을 높입니다. 초보자가 시작하기 가장 좋은 영역 중 하나입니다.



GROWTH

기능 추가 (New Features)

프로젝트에 새로운 가치를 더하는 기능을 구현합니다. 제안(Proposal) 단계부터 시작하여 설계 및 구현을 진행합니다.



OPTIMIZATION

성능 개선 (Performance)

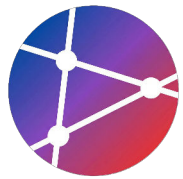
실행 속도를 높이거나 메모리 및 리소스 사용량을 최적화합니다. 알고리즘 개선이나 비효율적인 로직을 수정합니다.



QUALITY

리팩토링 (Refactoring)

외부 동작은 유지하면서 내부 코드 구조를 개선합니다. 가독성을 높이고 유지보수를 용이하게 만들어 기술 부채를 줄입니다.



| 기여 유형: 문서 기여

Documentation Contribution: The Gateway to Open Source



README 개선

프로젝트의 대문인 README 파일을 명확하고 최신 상태로 유지합니다. 설치 방법, 예제, 배지 추가 등 가독성을 높입니다.



API 문서 작성

함수, 클래스, 메서드에 대한 상세한 설명과 파라미터 정보를 작성하여 개발자들이 라이브러리를 쉽게 이해하고 사용하도록 돕습니다.



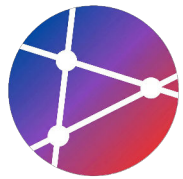
튜토리얼 작성

초보자를 위한 'Getting Started' 가이드나 특정 기능을 활용하는 실전 예제를 작성하여 진입 장벽을 낮춥니다.



번역 (Localization)

문서를 다양한 언어로 번역하여 전 세계 개발자들이 언어 장벽 없이 프로젝트에 접근할 수 있도록 기여합니다.



기여 유형: 이슈 관리

Contributions Beyond Code: Managing Issues & Communication



버그 리포트 (Bug Report)

소프트웨어의 오작동이나 예기치 않은 동작을 발견하여 상세히 보고합니다. 재현 단계(Reproduction Steps)와 환경 정보를 포함하는 것이 핵심입니다.



기능 제안 (Feature Request)

프로젝트의 발전을 위해 새로운 기능이나 개선 아이디어를 제안합니다. 구체적인 유즈 케이스와 기대 효과를 함께 제시하여 논의를 시작합니다.



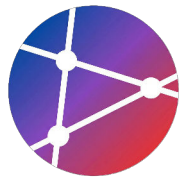
질문 답변 (Q&A)

다른 사용자의 질문에 답변하거나 기술적인 도움을 제공합니다. GitHub Discussions나 Issue 댓글을 통해 커뮤니티 활성화에 기여합니다.



이슈 트리아지 (Issue Triage)

접수된 이슈를 검토하여 중복을 제거하고, 라벨을 부착하며, 우선순위를 분류합니다. 메인테이너의 업무 부하를 줄여주는 중요한 기여입니다.



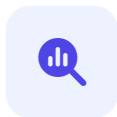
| 기여 유형: 리뷰

Contribution Type: Code Review & Feedback



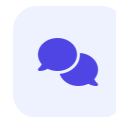
Pull Request 리뷰

다른 개발자의 코드를 검토하고 논리적 오류나 개선점을 찾아 프로젝트의 안정성을 높입니다.



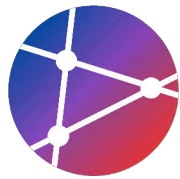
코드 품질 체크

코딩 컨벤션 준수 여부, 테스트 커버리지, 성능 이슈 등을 체계적으로 확인합니다.



제안 및 피드백

더 나은 구현 방법을 제안하고 건설적인 피드백을 통해 코드 품질을 지속적으로 개선합니다.



| 기여 유형: 커뮤니티

Expanding Impact: Forums, Events, and Knowledge Sharing



포럼 및 토론 참여

Stack Overflow, GitHub Discussions 등에서 질문에 답변하고 기술적 토론에 참여합니다. 초심자를 위한 멘토링 활동은 커뮤니티 성장의 핵심입니다.



이벤트 조직 및 참여

지역 밋업(Meetup), 컨퍼런스, 해커톤을 조직하거나 자원봉사자로 참여하여 오프라인 생태계를 활성화합니다.



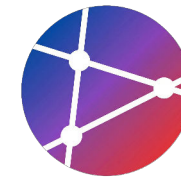
블로그 및 문서화

기술 블로그 작성, 튜토리얼 비디오 제작, 활용 사례 공유를 통해 프로젝트의 진입 장벽을 낮추고 지식을 전파합니다.



발표 및 강연

컨퍼런스 발표나 웨비나를 통해 프로젝트 사용 경험과 기술적 통찰력을 공유하고 프로젝트 인지도를 높입니다.



| 첫 기여 찾기

Finding Your First Contribution Opportunity

GitHub 직접 검색

🔍 Search: `label:"good first issue" is:issue is:open`

 'good first issue' 레이블은 메인테이너가 신규 기여자를 위해 특별히 남겨둔 비교적 쉬운 이슈입니다.

추천 큐레이션 사이트



Good First Issue

goodfirstissue.dev

인기 있는 오픈소스 프로젝트의 초보자용 이슈를 언어 별로 필터링하여 보여줍니다.



First Contributions

firstcontributions.github.io

실제 프로젝트에 기여하는 전체 흐름을 5분 만에 실습 해볼 수 있는 튜토리얼을 제공합니다.



Up For Grabs

up-for-grabs.net

초보자에게 도움을 요청하는 다양한 프로젝트 목록을 큐레이션하여 제공합니다.

| 기여 과정 (Contribution Process)

Open Source Contribution End-to-End Workflow



1



프로젝트 찾기

관심 분야 검색 및
Good First Issue 탐색

2



이슈 선택

이슈 내용 파악 및
참여 의사 댓글 작성

3



Fork & Clone

내 저장소로 Fork 후
로컬로 Clone

4



브랜치 생성

작업을 위한
새로운 브랜치 생성

5



코드 수정

코드 작성, 테스트 수행
및 가이드 준수

6



커밋 (Commit)

명확한 메시지와 함께
변경사항 저장

7



푸시 (Push)

내 원격 저장소(Origin)로
브랜치 업로드

8



Pull Request

원본 저장소로
병합 요청(PR) 생성

9



리뷰 대응

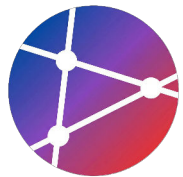
메인테이너 피드백 반영
및 코드 수정

10



머지 (Merge)

최종 병합 완료!
오픈소스 기여자 등극 🎉



| 기여 예시: Bug Fix PR

Example of a well-written Pull Request description

1 명확한 제목과 이슈 참조

PR 제목은 변경 사항을 즉시 알 수 있게 작성합니다. `Fixes #123` 과 같이 관련 이슈를 참조하면 머지 시 이슈가 자동 종료됩니다.

2 변경 사항 목록화

어떤 코드를 수정했는지, 검증 로직을 추가했는지 등 주요 변경점을 체크리스트나 불릿 포인트로 요약합니다.

3 테스트 및 스크린샷

어떤 테스트를 수행했는지 명시하고, UI 변경이 있다면 전/후 스크린샷을 첨부하여 리뷰어의 시간을 절약하세요.

✓ Review Tip

PR 내용이 상세할수록 리뷰어는 코드의 의도를 빨리 파악하고 더 빠르게 머지할 수 있습니다.

Markdown — Pull Request Template

```
## 🐛 Bug Fix
Fixes #123
## 📋 Changes
- Fixed validation error on empty email
- Added test case for empty input
- Updated error message constant

## 🧪 Testing
# Verified with the following test command:
```bash
npm test
```

> Test Suites: 5 passed, 5 total
> Tests:      24 passed, 24 total
## 📸 Screenshots
[Before/After Screen Capture Image]
```



SECTION 06



Inner Source

조직 내부의 오픈소스 방식

| Inner Source란?

Applying Open Source Culture Within the Organization

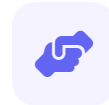


"Inner Source = 오픈소스 방식을 조직 내부에 적용"



내부 공개 (Internal Visibility)

코드는 조직 내부의 모든 구성원에게 공개되지만, 외부에는 비공개로 유지됩니다.



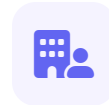
오픈소스 협업 방식

자발적 기여와 협업을 장려하는 오픈소스 문화를 사용합니다.



Pull Request & Code Review

모든 변경은 PR과 철저한 코드 리뷰를 통해 품질을 검증합니다.



부서 간 협업 (Breaking Silos)

부서 장벽을 제거하여 중복 개발을 막고 협업을 촉진합니다.

Inner Source vs Open Source

Comparison of Development Models and Characteristics



| 측면 | Open Source | Inner Source |
|------|-----------------------|-----------------------------|
| 가시성 | 전 세계 공개 (Public) | 조직 내부만 (Private / Internal) |
| 기여자 | 누구나 참여 가능 | 조직 구성원 (Employees) |
| 라이선스 | OSS 라이선스 (MIT, GPL 등) | 내부 정책 및 보안 규정 |
| 보안 | 커뮤니티에 의한 공개 검토 | 조직 내부 통제 및 관리 |
| 목적 | 커뮤니티 확장 및 생태계 | 조직 내 협업 및 효율성 |



Inner Source 이점

Benefits of Adopting Open Source Principles Internally



빠른 혁신 (Rapid Innovation)

부서 간 장벽을 제거하고 중복 작업을 최소화합니다. 기존 코드의 재사용성이 증가하여 시장 출시 시간을 단축시킵니다.



협업 문화 (Collaborative Culture)

조직 전반에 걸친 지식 공유와 멘토링이 활성화됩니다. 투명한 소통 구조를 통해 사일로(Silo) 현상을 방지합니다.



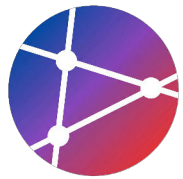
품질 향상 (Quality Improvement)

공개적인 코드 리뷰와 피드백 루프를 통해 버그를 조기 발견합니다. 내부 베스트 프랙티스가 자연스럽게 전파됩니다.



인재 육성 (Talent Growth)

개발자들이 다양한 내부 프로젝트에 참여하며 오픈소스 방식을 경험합니다. 이는 직무 만족도와 기술 역량을 높입니다.



Inner Source 구현: 문화 조성

Establishing the Cultural Foundation for Collaboration



오픈소스 마인드셋

"내 코드"가 아닌 "우리 코드"라는 주인의식을 공유합니다. 지식을 독점하지 않고 적극적으로 공유하며, 기여를 당연한 것으로 여기는 태도가 필요합니다.



협업 장려

부서 간 장벽(Silos)을 허물고 크로스 펑셔널(Cross-functional) 협업을 지원합니다. 다른 팀의 프로젝트에 기여하는 시간을 업무의 일부로 공식 인정해야 합니다.



실패 허용 (Psychological Safety)

비난 없는 포스트모템(Blameless Postmortem)을 실천하여 실패를 숨기지 않게 합니다. 실패를 학습과 개선의 기회로 삼아 혁신적인 시도를 장려합니다.

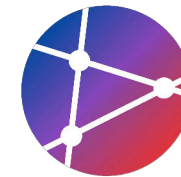


투명한 소통

모든 기술적 의사결정과 논의는 비공개 메신저가 아닌 공개된 이슈(Issue)와 PR에서 진행합니다. 정보의 비대칭을 최소화하여 누구나 문맥을 파악할 수 있게 합니다.

Inner Source 구현: 도구 준비

Essential Infrastructure for Successful Inner Source Adoption



FOUNDATION

VCS & Collaboration

GitHub Enterprise 또는 **GitLab**과 같은 중앙화된 플랫폼. 소스 코드 호스팅뿐만 아니라 Pull Request, 코드 리뷰, 이슈 트래킹을 통한 협업의 중심지 역할을 합니다.



SHARING

Internal Package Registry

사내에서 개발된 라이브러리와 모듈을 안전하게 배포하고 공유하기 위한 저장소. npm, Maven, PyPI 등의 사설 레지스트리를 구축하여 코드 재사용성을 극대화합니다.



AUTOMATION

CI/CD Pipeline

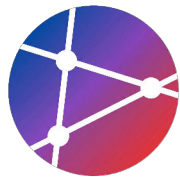
코드 변경 사항에 대한 자동화된 빌드, 테스트, 배포 시스템. **GitHub Actions**, **Jenkins** 등을 통해 품질 검증을 자동화하고 신속한 피드백 루프를 제공합니다.



KNOWLEDGE

Documentation Platform

프로젝트 가이드, API 문서, 기여 방법 등을 공유하는 지식 베이스. 검색 가능한 **Wiki**나 정적 사이트를 통해 진입 장벽을 낮추고 정보 접근성을 높입니다.



Inner Source 가이드라인

성공적인 Inner Source 도입을 위한 3대 핵심 정책



기여 프로세스

- ✓ 명확한 이슈/PR 템플릿 제공
- ✓ 브랜치 전략 표준화
- ✓ 커밋 메시지 규칙 (Conventional)



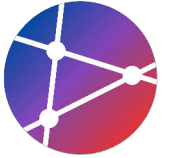
코드 리뷰 기준

- ✓ 린트(Lint) 및 스타일 자동화
- ✓ 필수 리뷰어 지정 및 승인 요건
- ✓ 테스트 커버리지 준수



라이선스/보안

- ✓ 내부 라이선스 정책 수립
- ✓ 의존성 패키지 보안 스캔
- ✓ Secret 키 포함 여부 점검



Inner Source 구현: 인센티브

지속적인 참여를 유도하기 위한 보상 및 인정 체계



기여 인정 (Recognition)

공개적인 칭찬, '이달의 기여자' 선정, 디지털 배지 부여 등을 통해 구성원의 노력을 가시적으로 인정하여 동기를 부여합니다.



커리어 발전 (Career)

오픈소스 기여 활동을 공식 인사 평가에 반영하고, 개인 포트폴리오로 활용할 수 있도록 지원하여 전문성 성장을 돕습니다.



학습 기회 (Learning)

기술 컨퍼런스 참가 지원, 외부 전문가 멘토링, 고급 교육 과정 제공 등 기여자를 위한 특별한 학습 및 네트워킹 기회를 제공합니다.



보상 체계 (Rewards)

금전적 인센티브뿐만 아니라 특별 휴가, 기념품(Swag), 팀 회식비 지원 등 실질적이고 다양한 보상을 통해 참여를 독려합니다.



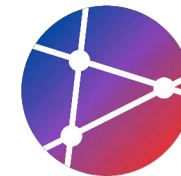
SECTION 07

오픈소스가 성과에 미치는 영향

DORA 연구와 성과 향상 요인

DORA 연구 결과

오픈소스/이너소스 기여 조직 vs 비기여 조직 성과 비교 (State of DevOps Report)



Lead Time for Changes

1.75x

↑ 더 빠름

코드 커밋에서 배포까지 소요 시간 단축



Deployment Frequency

1.5x

↑ 더 자주 배포

고객에게 가치를 전달하는 빈도 증가



Time to Restore

1.5x

↑ 복구 속도 향상

장애 발생 시 서비스 정상화 속도



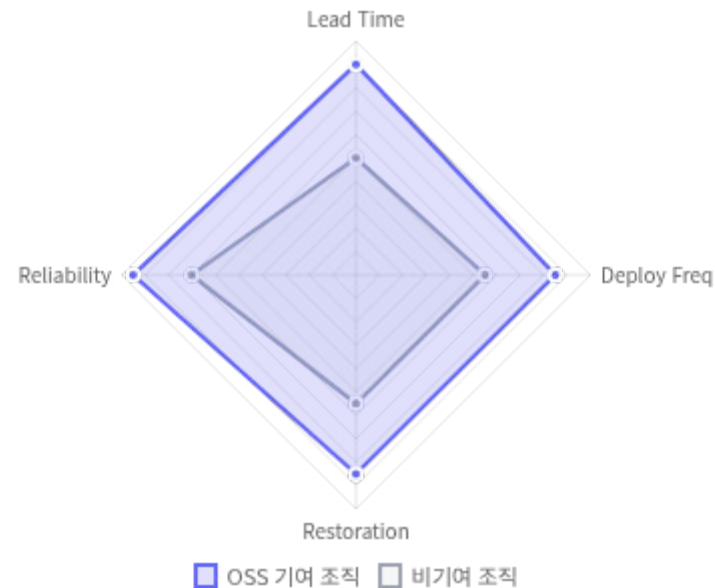
Change Failure Rate

-20%

↓ 장애율 감소

배포 시 실패할 확률의 유의미한 감소

종합 성과 비교



비즈니스 성과

- ✓ 시장 출시 시간(Time-to-Market) 획기적 단축
- ✓ 고객 피드백 반영 속도 증가로 만족도 향상
- ✓ 개발자 경험(DX) 개선 및 직원 만족도 증가



성과 향상 이유 1: 빠른 학습

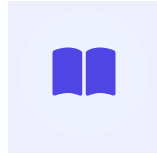
Rapid Learning & Skill Development

오픈소스 프로젝트 참여는 단순한 기여를 넘어, **개발자의 역량을 비약적으로 성장**시키는 학습의 장입니다.



다양한 코드베이스

전 세계의 다양한 아키텍처와
코드 스타일을 경험하며
시야를 확장합니다.



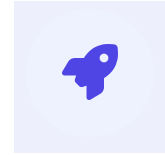
베스트 프랙티스

엄격한 코드 리뷰와
표준화된 개발 문화를 통해
모범 사례를 체득합니다.



스킬 향상

최신 기술 트렌드와
문제 해결 능력을
자연스럽게 습득합니다.



생산성 증가

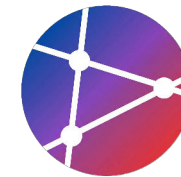
향상된 역량은
조직 전체의
퍼포먼스 향상으로
이어집니다.



| 성과 향상 이유: 품질 향상

How Open Source Improves Software Quality





성과 향상 이유: 혁신 가속

How Open Source Accelerates Innovation & Time-to-Market



검증된 라이브러리

수백만 개발자가 검증한
안정적인 오픈소스 컴포넌트를
즉시 활용하여 기반 구축



빠른 프로토타이핑

핵심 비즈니스 로직에만 집중하여
아이디어를 신속하게 구현하고
피드백 루프 단축



시장 출시 단축

개발 기간을 획기적으로 줄여
경쟁 우위를 확보하고
비즈니스 가치 조기 창출



오픈소스 생태계의 힘

"바퀴를 다시 발명하지 마라(Don't Reinvent the Wheel)." 오픈소스 생태계는 기업이 인프라 구축이 아닌 **차별화된 가치 창출**에 집중할 수 있게 해주는 혁신의 가속 페달입니다.



| 성과 향상 이유: 인재 유치

Talent Acquisition & Retention



개발자 만족도 증가

오픈소스 활동을 통해 개발자는 자율성을 느끼고, 커뮤니티와 소통하며 **직무 만족도**가 크게 향상됩니다.



우수 인재 유치

오픈소스 친화적인 기업은 혁신적인 이미지를 구축하여, **열정적이고 숙련된 개발자**들이 선호하는 직장이 됩니다.



이직률 감소

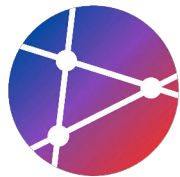
조직 내에서 성장 기회를 찾고 업무에 몰입하게 되어, 핵심 인재의 이탈을 방지하고 **장기 근속**을 유도합니다.



SECTION 08

실습 — 오픈소스 프로젝트 만들기

Step-by-Step 실습



실습 Step 1: 프로젝트 초기화

Initializing the repository and creating essential documentation

1 디렉토리 생성 및 Git 초기화

프로젝트를 위한 폴더를 생성하고 `git init` 명령어로 버전 관리를 시작합니다.

2 README.md 작성 (Heredoc)

Heredoc 문법(`<< 'EOF'`)을 사용하여 여러 줄의 콘텐츠를 한 번에 작성합니다.

3 필수 정보 포함

프로젝트의 기능(Features), 설치 방법(Installation), 사용법(Usage), 라이선스(License) 정보를 반드시 포함해야 합니다.

> Command Line Tip

터미널에서 텍스트 파일을 빠르게 생성할 때 `cat`과 `EOF`를 활용하면 편집기를 열지 않고도 작업할 수 있습니다.

```
bash — awesome-calculator

# 1. 레포지토리 생성
mkdir awesome-calculator
cd awesome-calculator
git init

# 2. README 작성
cat > README.md << 'EOF'
# Awesome Calculator

A simple but powerful calculator library.

## Features
- Basic arithmetic operations
- Scientific functions
- History tracking

## Installation
`` bash
npm install awesome-calculator
```



실습 Step 2: CONTRIBUTING.md

Creating a comprehensive contribution guideline for collaborators

1 Development Setup

기여자가 프로젝트를 로컬에서 실행하고 테스트하는 방법을 안내합니다. `npm install`, `npm test` 등이 포함됩니다.

2 Submitting Changes

기여 프로세스를 명확히 정의합니다. Fork → Branch → Commit → PR 순서를 명시하여 혼란을 방지하세요.

3 Code Style

프로젝트의 코딩 컨벤션을 설명합니다. 들여쓰기, 세미콜론 사용 여부, Linter 규칙 등을 포함하여 코드 일관성을 유지합니다.

Why it matters

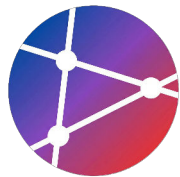
명확한 기여 가이드라인은 진입 장벽을 낮추고 양질의 PR을 유도하는 가장 효과적인 방법입니다.

```
bash — CONTRIBUTING.md

cat > CONTRIBUTING.md << 'EOF'
# Contributing
## Development Setup
```bash
npm install
npm test
```

## Submitting Changes
1. Fork the repo
2. Create a branch
3. Make your changes
4. Run tests
5. Submit PR

## Code Style
- ESLint configuration
- 2 spaces indentation
- Semicolons required
```

실습 Step 3-4: 코드 & 테스트

Implementing core logic and writing unit tests with Jest

3 핵심 로직 구현 (index.js)

`Calculator` 클래스를 정의하고 4칙 연산 메서드를 구현합니다. 특히 나눗셈에서 0으로 나누는 경우에 대한 예외 처리가 중요합니다.

4 테스트 작성 (test/calculator.test.js)

작성한 로직이 올바르게 동작하는지 `Jest` 를 사용하여 검증합니다. 정상적인 연산 결과와 예외 상황을 모두 테스트하세요.

✓ Best Practice

테스트 코드는 문서로서의 역할도 합니다. `describe` 와 `test` 설명을 명확하게 작성하여 코드의 의도를 전달하세요.

```
VS Code - index.js & calculator.test.js

// 1. index.js
class Calculator {
  add(a, b) {
    return a + b;
  }

  subtract(a, b) {
    return a - b;
  }

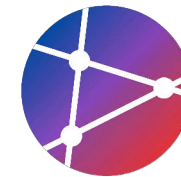
  multiply(a, b) {
    return a * b;
  }

  divide(a, b) {
    if (b === 0) throw new Error('Division by zero');
    return a / b;
  }
}

module.exports = Calculator;
```

CI/CD 및 배포 설정

GitHub Actions automation & Publishing to npm registry



5 GitHub Actions 자동화

`.github/workflows/test.yml` 파일을 생성하여 Push 및 Pull Request 시 자동으로 테스트가 실행되도록 설정합니다.

6 패키지 설정 (package.json)

`name` (유일성 확보), `version`, `main` 등 배포에 필요한 필수 메타 데이터를 확인합니다.

7 배포 (Publish)

npm 레지스트리에 로그인하고 패키지를 배포합니다. 이 과정은 전 세계 개발자에게 코드를 공유하는 순간입니다.

Action

배포 전 반드시 `npm test` 를 로컬에서 통과해야 합니다.

```
code — CI/CD & Config

# 1. GitHub Actions Workflow (.github/workflows/test.yml)
name: Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
    with:
      node-version: '18'
      - run: npm install
      - run: npm test

# 2. package.json Configuration
{
  "name": "awesome-calculator",
  "version": "1.0.0",
  "main": "index.js",
```

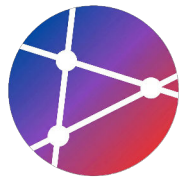


SECTION 09



실습 과제

4개의 과제 수행 및 제출



과제 1: 오픈소스 프로젝트 생성

자신만의 오픈소스 프로젝트를 시작하고 필수 요소를 갖추습니다.

수행 체크리스트

- ✓ **GitHub Public 레포지토리 생성**
적절한 이름과 설명을 포함하여 공개 저장소를 생성합니다.
- ✓ **필수 문서 작성 (README, LICENSE)**
MIT 라이선스를 적용하고, 프로젝트를 설명하는 README.md를 작성합니다.
- ✓ **기여 가이드 및 행동 강령 추가**
CONTRIBUTING.md와 CODE_OF_CONDUCT.md 파일을 추가하여 협업 기반을 마련합니다.
- ✓ **첫 릴리스 배포**
버전 태그(v1.0.0)를 생성하고 릴리스 노트를 작성합니다.

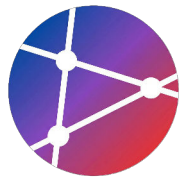
제출 정보

FORMAT GitHub 레포지토리 URL 제출
DEADLINE 다음 주 수업 시작 전까지 (LMS 업로드)

평가 포인트

- 모든 필수 파일(4종) 존재 여부
- README 문서의 완성도 및 가독성
- 라이선스 적용의 정확성
- 커밋 메시지의 명확성

💡 **Tip:** `.gitignore` 파일을 잊지 마세요! 불필요한 시스템 파일이 올라가지 않도록 주의해야 합니다.



과제 2: 오픈소스 기여

실제 오픈소스 프로젝트의 이슈를 찾아 해결하고 PR을 제출하는 전체 과정을 경험합니다.

기여 프로세스 체크리스트



Good First Issue 찾기

'good first issue' 레이블이 붙은 이슈를 검색하거나 추천 사이트를 활용합니다.



Fork & Clone

프로젝트를 자신의 계정으로 포크하고 로컬 환경에 클론합니다.



이슈 해결 및 커밋

새 브랜치를 생성하고 코드를 수정합니다. 테스트 통과는 필수입니다.



Pull Request 제출

PR 템플릿에 맞춰 변경 사항을 상세히 작성하고 제출합니다.



리뷰 대응 및 머지

메인테이너의 피드백을 반영하고 최종 머지(또는 승인)를 확인합니다.

제출 정보

FORMAT PR 링크 (Merged 또는 Open 상태)

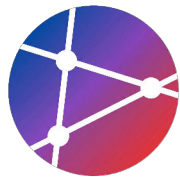
DEADLINE 학기 말 프로젝트 발표 전까지

평가 포인트

- 적절한 난이도의 이슈 선택
- PR 설명의 구체성 및 템플릿 준수
- 커밋 메시지 규칙(Conventional Commits) 준수
- 리뷰어와의 소통 태도



Tip: 코드 수정이 부담스럽다면, 문서(Docs) 수정이나 오타 수정부터 시작해보세요. 모든 기여는 소중합니다!



| 과제 3: 라이선스 분석

실제 성공한 오픈소스 프로젝트들의 라이선스 전략을 분석하고 인사이트를 도출합니다.

☰ 수행 체크리스트

- ✓ **유명 오픈소스 프로젝트 5개 선정**
다양한 분야(웹, AI, OS 등)의 대표적인 오픈소스 프로젝트를 선정합니다.
- ✓ **각 프로젝트의 라이선스 조사**
GitHub 저장소의 LICENSE 파일을 확인하여 정확한 라이선스 종류를 파악합니다.
- ✓ **라이선스 선택 이유 분석**
해당 프로젝트가 왜 그 라이선스를 선택했는지(기업 전략, 커뮤니티 보호 등) 분석합니다.
- ✓ **분석 보고서 작성**
조사 내용과 분석 결과, 시사점을 포함한 보고서를 작성합니다.

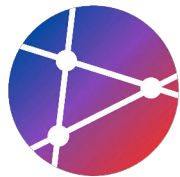
📌 제출 정보

FORMAT PDF 보고서 (자유 양식, 2-3페이지)
DEADLINE 다음 주 수업 시작 전까지 (LMS 제출)

☑ 평가 포인트

- 프로젝트 선정의 다양성 (Permissive vs Copyleft)
- 라이선스 특징에 대한 정확한 이해도
- 선택 이유 분석의 논리적 타당성
- 보고서의 구성 및 가독성

💡 **Tip:** `Choose a License` 사이트나 `tldrlegal.com`을 참고하면 복잡한 법적 용어를 쉽게 이해할 수 있습니다.



과제 4: Inner Source 계획

조직 내 오픈소스 문화를 도입하기 위한 전략과 실행 계획을 수립합니다.

수행 체크리스트

- ✓ **조직 내 Inner Source 도입 계획 수립**
도입 대상 부서나 시범 프로젝트를 선정하고 적용 범위를 정의합니다.
- ✓ **예상 이점 및 도전 과제 분석**
조직 상황을 바탕으로 기대 효과(이점)와 잠재적 장애요인(도전 과제)을 분석합니다.
- ✓ **실행 로드맵 작성**
문화 조성, 도구 도입, 파일럿 운영 등 단계별 실행 계획을 수립합니다.
- ✓ **핵심 성과 지표(KPI) 설정**
참여도, 코드 재사용률 등 성공 여부를 측정할 구체적인 지표를 제안합니다.

제출 정보

FORMAT PDF 보고서 (3-5 pages)

DEADLINE 다음 주 수업 시작 전까지 (LMS 업로드)

평가 포인트

- 현실적인 도입 시나리오 여부
- 장애물에 대한 구체적 해결책 제시
- 측정 가능한 성과 지표(KPI) 설정
- 기존 조직 문화를 고려한 접근

💡 **Tip:** 'InnerSource Commons'의 실제 기업 도입 사례(Case Studies)를 참고하면 더욱 현실적인 계획을 세울 수 있습니다.



SECTION 10

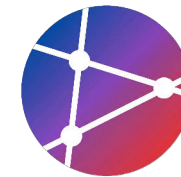


참고 자료

공식 가이드·도구·도서

참고 자료

Essential Reading & Finding Open Source Projects



필수 읽기

오픈소스 이해를 위한 필독서 및 가이드



Open Source Guide

오픈소스 프로젝트를 만들고 유지 관리하는 방법에 대한 포괄적인 가이드 (opensource.guide)



Choose a License

상황별로 프로젝트에 가장 적합한 라이선스를 선택하도록 돕는 도구 (choosealicense.com)



The Cathedral and the Bazaar

오픈소스 개발 모델의 철학과 효율성을 설명한 에릭 레이먼드의 고전 에세이



오픈소스 찾기

기여할 프로젝트를 찾기 위한 플랫폼



GitHub Explore

트렌딩 레포지토리, 토픽별 인기 프로젝트 및 개인화된 추천 탐색



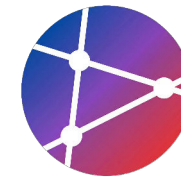
Good First Issue

초보 개발자가 기여하기 좋은 'good first issue' 레이블이 붙은 이슈 큐레이션



First Timers Only

오픈소스에 처음 기여하는 사람들을 위한 친절한 가이드와 리소스 모음



참고 자료: Inner Source & Books

심화 학습을 위한 Inner Source 리소스와 추천 도서 목록



COMMUNITY

InnerSource Commons

세계 최대의 Inner Source 커뮤니티이자 지식 공유 플랫폼입니다. 다양한 패턴과 사례 연구를 제공합니다.

innersourcecommons.org



BOOK

Working in Public

Nadia Eghbal

오픈소스 소프트웨어의 유지보수와 현대적 개발 문화에 대한 깊이 있는 통찰을 제공합니다.



TOOLKIT

InnerSource Patterns

조직 내 Inner Source 도입 및 운영 시 발생하는 문제들을 해결하기 위한 검증된 솔루션 모음입니다.

patterns.innersourcecommons.org



BOOK

The Success of Open Source

Steven Weber

오픈소스의 경제적, 사회적 성공 요인과 독특한 협업 모델을 학술적으로 분석한 필독서입니다.



BOOK

Producing Open Source Software - Karl Fogel

오픈소스 프로젝트의 시작부터 운영, 커뮤니티 관리까지 실질적인 가이드를 제공하는 개발자와 관리자를 위한 지침서입니다.



SECTION 11

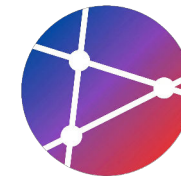


핵심 요약

Key Takeaways와 다음 단계

핵심 요약

Key Takeaways, Etiquette & Next Steps



기억해야 할 것

- ✓ **오픈소스 3요소**
투명성 + 협업 + 혁신
- ✓ **라이선스 선택**
신중하게 선택 (일반적으로 MIT)
- ✓ **기여의 다양성**
코드뿐만 아니라 문서, 이슈, 리뷰도 중요
- ✓ **Inner Source**
내부 오픈소스 문화로 조직 혁신
- ✓ **성과 향상**
오픈소스 참여는 개발 및 비즈니스 성과 직결



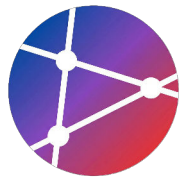
오픈소스 에티켓

- 🙏 **존중과 감사**
메인테이너와 기여자를 존중
- 📖 **문서 정독 (RTFM)**
질문 전 문서와 기존 이슈 먼저 검색
- 🐛 **명확한 리포트**
재현 가능한 단계와 환경 정보 제공
- 💬 **정중한 소통**
건설적이고 예의 바른 언어 사용
- 🕒 **인내심**
오픈소스는 대부분 자원봉사이므로 기다림 필요



다음 단계

- ▶ **Project Start**
첫 오픈소스 프로젝트 레포지토리 생성
- ▶ **Contribution**
관심 있는 외부 프로젝트에 'Good First Issue' 기여
- ▶ **Inner Source**
조직 내 Inner Source 도입 계획 및 제안
- ▶ **Community**
오픈소스 커뮤니티 활동 및 네트워킹 참여



다음 주차 예고 및 마무리

Next Week Preview & Conclusion



다음 주차 안내

다음 주차의 구체적인 주제와 일정은 **LMS 공지사항**을 통해 안내될 예정입니다.



과제 피드백

제출해주신 실습 과제에 대한 **개별 및 종합 피드백 요약**이 다음 수업 전 제공됩니다.



사전 학습

원활한 실습 진행을 위해 공지된 **사전 읽기 자료 링크**를 반드시 확인해주세요.

“

"If I have seen further, it is by standing on the shoulders of giants."

— Isaac Newton

오픈소스는 우리 모두의 어깨 위에 서 있습니다. ✨